



Projecto de Programac,ao com Objectos 17 de Setembro de 2024

Essay on Duvidas:

- Consulting services: Presence and através do endec,o po-tagus@disciplinas.tecnico.ulisboa.pt.
- Nao utilizar fonts de informac,ao n,ao oficialmente associadas ao corpo docente (podem colocar em causa a aprovac,ao a disciplina).
- Nao s,ao aceites justificac,oes para violac,oes destes conselhos: quaisquer consequencias nefastas,ao da responsabilidade do aluno.

Requisitos para desenvolvimento, material de apoio e actualizac,oes do enunciado (ver informac,ao completa na secc,ao [Projecto] no Fenix):

- O material de apoio e de uso obrigatorio en ao pode ser alterado.
- Verificar atempadamente (mínimo de 48 horas antes do final de cada prazo) os requisitos exigidos pelo processo de avaliacao,ao.

Processo de avaliacao,ao (ver informac,ao completa nas secc,oes [Projecto] e [Metodo de Avaliacao,ao] no Fenix):

- Datas: Ver secc,ao Projecto da pagina da disciplina para as 3 entregas do projecto . Teste prático Consultar a mesma secc,ao.
- Nao ser,ao consideradas quaisquer alterac,oes aos ficheiros de apoio disponibilizados: eventuais entregas dessas alterac,oes substituídas durante avaliacao,ao oes ser automaticamente da funcionalidade do código entregue.
- Trabalhos n,ao entregues no Fenix at e final do prazo t em classificac,ao 0 (zero) (n,ao s,ao aceites outras formas de entrega).
- A Valiacao,ao do projecto press and a compromisso de Honra de Que o trabalho foi realizado pelos alunos correspondentes ao grupo de Avaliacao,ao.
- Fraudulent executions of project projects andao como results .

The objective and the desenvolver uma aplicacao,ao de gestao de um hotel veterinario. secc,ao 1 apresenta as entidades do domínio da aplicacao,ao a desenvolver. As funcionalidades da aplicacao,ao a desenvolver

Sao descritas nas secc,oes 4 e 5. A secc,ao 2 descreve os requisitos de desenho que aplicacao,ao desenvolve deve oferecer.

1 Entities of the House

Nesta secc,ao descrevem-se as varias entidades que vao ser manipuladas no contexto da aplicacao,ao a desenvolver. Existem varios conceitos importantes neste contexto (por exemplo, hotel veterinario, especie, animal, funcionario).

Um hotel veterinary tem varios habitats on os animais colocados, a guarda do hotel s,ao depois colocados. Cada animal pertence a uma especie. O hotel tem dois tipos de funcionarios: tratadores e veterinarios. e pela limpeza dos habitats.

O habitat pode ainda ter arvores. Deve ser possível calcular o grau de satisfacao dos animais e dos funcionarios.

It's the most important thing, it's different, it's different, it's different, it's different, it's different, it's different, it's irrelevant.

Na concretizacao desta aplicacao,ao poder,ao considerar outros conceitos al em dos que s,ao identificados explicitamente a ser necess nesta secc,ao.

1.1 Especies

Cada animal pertence a uma dada especie. n,ao existem

The difference between the species is the distintas com o mesmo nome) and the connotations are os animais que pertencem ecie. a esp

1.2 Animais

Os animais sao identificaveis por uma chave unica (cadeia de caracteres arbitrary aria definida na altura da criacao,ao). ao sobre o seu estado de saude.

E possível calcular o nível de satisfacao de um animal. Este valor depende varios factores.

animais de outras espécies.
os animais que lá vivem.

$$\text{satisfac}_{ao}(a) = 20 + 3 \cdot \text{especieIgual}(a, h) \cdot \frac{\text{area}}{\text{populac}_{ao}(h)} + \text{especieDiferente}(a, h) + \text{adequac}_{ao}(a, h) \quad (1)$$

onde $\text{especieIgual}(a, h)$ representa o número de animais da mesma espécie de a presentes em h (sem contar com a), $\text{especieDiferente}(a, h)$ representa o número de animais presentes em h de espécies diferentes da de a , $\text{populac}_{ao}(h)$ representa o número de animais no habitat h , $\text{adequac}_{ao}(a, h)$ representa influência do habitat no animal tendo em conta a espécie do animal. A adequac_{ao} é 20 se a influência do habitat for positiva, ≤ 0 se for negativa e 0 se a influência for neutra (valor por omissão).

1.3 Habitats

Os habitats são identificados por uma chave única (cadeia de caracteres definida na altura do registo). Cada habitat tem um nome (cadeia de caracteres não única) e uma área (n número inteiro). Os habitats conhecem ainda as árvores e os animais presentes no habitat.

Os habitats podem ser mais (ou menos) adequados para determinadas espécies. A adequac_{ao} tem impacto no grau de satisfac_{ao} dos animais. Por omissão, um habitat tem um impacto neutro para cada espécie.

1.4 Árvores

As árvores são identificadas por uma chave única (cadeia de caracteres arbitrária definida na altura da criação, ao). Cada árvore tem ainda nome (cadeia de caracteres não única) e idade em anos (n número inteiro). Quando as árvores são sincronizadas

uma árvore é criada, fica na estação ao do ano em que a aplicação estiver no momento, i.e., todas as nesse aspecto. as árvores estão

Assim, por exemplo, as árvores de folha caduca perdem as folhas principalmente durante o Outono, ficando sem folhas no Inverno. as árvores, mas mais durante o Inverno.

Árvore

O primeiro corresponde a uma base de limpeza da árvore (designado como dificuldade de limpeza). O segundo, designado como esforço sazonal, depende da estação do ano e do tipo de árvore, como indicado na tabela seguinte:

Estação \ Árvore	Inverno	Primavera	Verão	Outono
Folha caduca	0		-	5
Folha perene	2	1	1	1

O terceiro corresponde a um factor que cresce logaritmicamente com a idade da árvore: $\log(\text{idade} + 1)$ (logaritmo natural). Combinações de factores, factores que podem afectar a sua saúde da árvore são calculados pela seguinte fórmula:

$$\text{esforço_limpeza}(a) = \text{dificuldade_limpeza}(a) \cdot \text{esforço_sazonal}(a) \cdot \log(\text{idade}(a) + 1) \quad (2)$$

As árvores envelhecem com as estações, i.e., a idade aumenta em 1 unidade (1 ano) a cada 4 estações (note-se que o incremento não é necessariamente simultâneo em todas as árvores).

1.5 Funcionários

Os funcionários são identificados por uma chave única (cadeia de caracteres arbitrária definida na altura da criação, ao). Os funcionários têm um nome (cadeia de caracteres não única) e veterinários. Cada tratador está atribuído a um conjunto de habitats (zero ou mais). satisfac_{ao} . O cálculo da satisfac_{ao} depende do tipo de funcionário.

1.5.1 Satisfação dos Veterinários

A satisfação de um veterinário é afetada positivamente pela existência de outros veterinários que sabem vacinar as várias

The difference between the species and the vaccine is the negative value of the animals and the response to the disease (ou seja, que o veterinário sabe vacinar).

$$\text{satisfação}(v) = 20 - \frac{\text{população}(e)}{\text{número de veterinários}(v)(e)} \quad (3)$$

onde $\text{espécies}(v)$ representa o conjunto de espécies que o veterinário sabe tratar, $\text{população}(e)$ representa o número de animais da espécie e $\text{número de veterinários}(e)$ representa o número de veterinários com a responsabilidade de vacinar a espécie v .

1.5.2 Satisfação dos Tratadores

A satisfação dos tratadores é afetada negativamente pelo trabalho que lhes é atribuído. O trabalho depende dos vários habitats atribuídos ao tratador.

$$\text{satisfação}(t) = 300 - \frac{\text{trabalho no habitat}(h)}{\text{número de habitats}(t)} \quad (4)$$

$$\text{trabalho no habitat}(h) = \text{área}(h) + 3 - \text{população}(h) + \text{esforço de limpeza}(h) \quad (5)$$

onde $\text{habitats}(t)$ representa o conjunto de habitats sob responsabilidade do tratador t e $\text{árvores}(h)$ representa o conjunto de árvores presente no habitat h .

1.6 Vacinas

Cada vacina tem um nome (cadeia de caracteres; não única) e sabe as espécies a que pode ser aplicada. Cada vacina tem um nome (cadeia de caracteres; não única) Mantém ainda um registo de todas as vezes que foi administrada.

1.6.1 Problemas associados à vacinação

O estado de saúde de um animal é o histórico de eventos (inicialmente vazio) relacionados com a vacinação do animal, entre os quais estão os danos causados por várias vacinações. O valor do dano de uma vacinação correcta (a espécie do animal está incluída nas espécies para as quais a vacina é apropriada) é igual a 0. O valor do dano causado pela administração de uma vacina a um animal depende do número de letras distintas entre o nome da espécie a e o nome das espécies a que se destinava v ($\text{espécies}(v)$):

$$\text{dano}(v, a) = \max_{e \in \text{espécies}(v)} (\text{tamanho}(\text{nomes}(\text{espécie}(a), e) - \text{caracteres comuns}(\text{espécie}(a), e))) \quad (6)$$

em que

$$\text{tamanho}(\text{nomes}(\text{espécie1}, \text{espécie2})) = \max(\text{tamanho}(\text{espécie1}), \text{tamanho}(\text{espécie2})) \quad (7)$$

Considere-se que os nomes a e b têm apenas 1 carácter distinto, sendo a ordem irrelevante. 6, sendo calculado por: $\max(\text{tamanho}(a), \text{tamanho}(b)) - \text{caracteres comuns}(a, b) = 8 - 2 = 6$

A seguinte tabela descreve os resultados associados ao dano causado pelas vacinações:

Dano	Termo adicional na apresentação do estado de saúde
0 (mesma espécie)	NORMAL
1 a 4 (espécies diferentes)	CONFUSÃO
5 ou mais	ACIDENTE
	ERRO

The results are great, the animals are the best 2 vaccinates 1 and 4 (ACIDENTE), the vaccines are 1 and 4 (ACIDENTE), the vaccinators are 1 and 4 (ACIDENTE), the vaccinator is the superior 5 (ERRO) and the vaccinator is the correct one, the present is the history of health (usa-se) avigula como separator): ACIDENTE,ACIDENTE,ERRO,ERRO,ERRO,NORMAL (ie, concatenação ordenada por ordem de ocorrência).

1.7 Serialization

É possível guardar e recuperar o estado actual da aplicação, preservando toda a informação relevante do domínio da aplicação. A serialização Java usa as classes da package `java.io`, em particular, a `interface java.io.Serializable` e as classes de leitura `java.io.ObjectInputStream` e escrita `java.io.ObjectOutputStream` (entre outras).

2 Requirements for Desenho

O sistema a desenvolver deve seguir o princípio de desenho aberto-fechado por forma a aumentar a extensibilidade do seu código. Por exemplo, deve ser possível adicionar novos tipos de funcionários com um impacto mínimo no código já existente do domínio da aplicação.

Aplicação de um padrão de desenho para melhorar a legibilidade do código relacionado com o conceito árvore e permitir ao usuário ter uma boa experiência, não importa o que aconteça, relacionados com as well as estas coisas do ano sem que isso implique alterar código relacionado com árvore. The value of uma árvore, que vai depender do tipo de árvore e da estação actual) sem comprometer a legibilidade da solução.

Aplicação de um padrão de desenho que permita novas políticas de cálculo da satisfação dos funcionários sem impactar no código existente. tempo de execução. Quando for necessário, a aplicação deve ser capaz de criar um funcionário, o cálculo da satisfação segue a fórmula descrita neste enunciado.

Apesar de haver na especificação actual apenas um hotel, o código não deve assumir que não possam vir a existir mais instâncias simultâneas.

Do your best to be a realist and a primeira entrega do projecto não é necessário ter em conta estes dois requisitos. Estes dois requisitos apenas precisam de ser considerados na entrega final do projecto.

3 Arquitectura da Aplicação

A aplicação a desenvolver deve seguir uma arquitectura em camadas: as entidades relacionadas com a interacção com o utilizador estão concretizadas na camada de apresentação (package `hva.app`) e as entidades relacionadas com as entidades do domínio estão concretizadas na camada de domínio (package `hva.core`).

da aplicação não deve haver código de interacção com o utilizador no código respeitante ao domínio da aplicação (concretizada na camada de domínio).

Interface com o utilizador sem que isso implique alterar o código da camada de domínio. Para garantir um melhor desenho da aplicação toda a lógica de negócio deve estar concretizada no código respeitante ao domínio da aplicação (camada `core`) e não na camada de apresentação. manter.

A framework of interacção com o utilizador deve ser realizada utilizando a framework of interacção com o utilizador **po-UILib** (disponível na secção **Projecto** da página da disciplina. `pt.technico.po.ui.Menu`) (fornecida pela framework), onde se deve indicar qual o conjunto de opções do menu. Cada opção

um menu (designado como comando) e concretizada por uma subclasse de `pt.technico.UILib.Menus.Command` (também fornecida pela framework) que tem a responsabilidade de suportar a funcionalidade correspondente a opção em causa. Todos os menus suportam automaticamente a opção **Sair** (fecha o menu).

As operações de pedido e apresentação de informação ao utilizador **devem** realizar-se através dos objectos `form` e `display`, respectivamente, presente em cada comando. interfaces já definidas na camada de apresentação do domínio a criar pelo menos mais uma instância de

Esqueleto da aplicação fornecido (ver secção 3.1).

It's a good idea to be in control of your situation. It's the best time to do it. deve instanciar uma excepção

e lançar a seguinte. Estas excepções serão depois tratadas automaticamente pela classe `Menu`. Lançar a seguinte. Estas excepções serão depois tratadas automaticamente pela classe

3.1 Esqueleto da Aplicação

It's a great experience, it's a great deal, it's a great deal, it's a great deal, it's a great deal. da cadeira. O esqueleto inclui

a class `hva.app.App`, que representa o ponto de entrada da aplicação, ao desenvolver e os vários comandos e menus da aplicação, ao desenvolver (descritos na secção 4). Alguns dos comandos já estão ao completamente finalizados, outros (a grande maioria) estão ao Parcial concretizados and tem ser alterados por forma a suportarem a funcionalidade descrita neste enunciado. Cada menu respectivos comandos estão definidos num subpackage de `hva.app`. Por exemplo, o menu principal e respectivos comandos ESTÃO definidos em `hva.app.main`. Cada menu está completamente concretizado e a grande maioria dos comandos apenas tem o esqueleto da classe e necessário acrescentar código por forma a que o comando tenha a funcionalidade necessária.

Cada subpackage relacionado com um menu contém ainda as interfaces `Message`, `Prompt` (definem as mensagens a utilizar). It's the best way to use your product. **Não** podem ser definidas novas Mensagens. Potenciais omissões development ser esclarecidas com o corpo docente antes de qualquer concretização.

O esqueleto inclui ainda algumas classes do domínio da aplicação (presentes no package `hva.core`) parcialmente concretizadas. Será ainda necessário concretizar novas entidades do domínio da aplicação e finalizar as que já estão parcialmente fornecidas por forma a terem conjunto de entidades que oferecem a funcionalidade do domínio da aplicação a desenvolver.

Finalmente, as exceções a lançar nos vários comandos já estão definidas no package `hva.app.exception` do esqueleto da aplicação. **Não** podem ser alteradas e apenas devem ser utilizadas no código de interação com o utilizador pela razão indicada anteriormente.

mas podem (e devem) ser acrescentadas novas exceções para representar outras situações contextuais, desde que possam ocorrer no âmbito das entidades do domínio.

4 Interação com o utilizador

Esta secção descreve a **funcionalidade máxima** da interface com o utilizador, antes de proceder à sua validação (exceto onde indicado).

A aplicação permite gerir informação sobre as várias entidades do modelo. Inicialmente, a aplicação está vazia, mas pode ser carregada uma base de dados textual com conceitos pre-definidos. Neste caso, a aplicação com estado referente às

entidades que foram carregadas na aplicação. Deve ser possível registar, visualizar, e operar sobre as várias entidades do domínio. diferentes entidades geridas pela aplicação. A apresentação de listas (eg, animais, funcionários, etc.) faz-se por ordem crescente da respectiva chave, exceto em caso de indicação em contrário. A ordem das chaves alfanuméricas deve ser lexicográfica (UTF-8), **nao havendo** distinção entre maiúsculas e minúsculas.

Para cada interação com o utilizador que exista no contexto de um comando em que utilizar uma mensagem específica e necessária.

Indicando qual e onde a interface deve ser utilizada, por exemplo, `prompt` ou `message` que é a resposta no contexto de um comando. Os identificadores de objetos do domínio necessários para a realização da funcionalidade do comando em causa.

The difference between the person and the person is the object of the conhecido: ao em contrário

Entidade	Pedido a apresentar	Exceção a lançar se desconhecido
Animal	<code>hva.app.animal.Prompt.animalKey()</code>	<code>UnknownAnimalKeyException</code>
Especie	<code>hva.app.animal.Prompt.speciesKey()</code>	<code>UnknownSpeciesKeyException</code>
Funcionario	<code>hva.app.employee.Prompt.employeeKey()</code>	<code>UnknownEmployeeKeyException</code>
Habitat	<code>hva.app.habitat.Prompt.habitatKey()</code>	<code>UnknownHabitatKeyException</code>
Arvore	<code>hva.app.habitat.Prompt.treeKey()</code>	<code>UnknownTreeKeyException</code>
Vacina	<code>hva.app.vaccine.Prompt.vaccineKey()</code>	<code>UnknownVaccineKeyException</code>

Note-se que estas exceções não devem ser utilizadas no código do domínio da aplicação como explicado em 3. Alguns casos particulares podem usar pedidos específicos não apresentados nesta tabela.

4.1 Menu Principal

As ações deste menu permitem gerir a salvaguarda do estado da aplicação, abrir submenus e aceder a alguma informação global.

A list of completa e a seguinte: **Ver Estado do Sistema, Gestão ciclo biológico das Gestao de arvores, de Animais, the funniest things to do as, Vacinas Gestao de Habitats e Consultas.**

Mensagens de algo para este menu estão definidos em `hva.app.main.Prompt` e `hva.app.main.Message`.

Estes comandos estão concretizados (nalguns casos parcialmente) nas seguintes classes do package `hva.app.main`: `DoNewFile`, `DoOpenFile`, `DoSaveFile`, `DoAdvanceSeason`, `DoShowGlobalSatisfaction`, `DoOpenAnimalsMenu`, `DoOpenEmployeesMenu`, `DoOpenHabitatsMenu`, `DoOpenVaccinesMenu` e `DoOpenLookupsMenu`.

4.1.1 Salvaguarda do estado actual

Inicialmente, a aplicação a vazia ou tem apenas informação sobre as entidades que foram carregados no arranque via ficheiro textual (ver 5).-ao) Pode ser guardado para posterior recuperação (via serialização Java: `java.io.Serializable`). Na leitura e escrita do estado da aplicação,

Devem ser tratadas as exceções associadas. A funcionalidade é a seguinte:

Criar – Cria uma nova aplicação a vazia: a aplicação não fica associada a nenhum ficheiro (é anónima).

Abrir – Carrega os dados de uma sessão anterior a partir um ficheiro previamente guardado (ficando este ficheiro associado aplicação, para futuras operações de salvaguarda). devolvida por `Prompt.openFile()`.

lançada a excepção `FileNotFoundException`. A execução bem sucedida desta opção substitui toda a informação relevante da aplicação.

Guardar – Guarda o estado actual da aplicação no ficheiro associado. aplicação associada

ada a este ficheiro. Não é executada nenhuma acção se não existirem alterações desde a última salvaguarda.

Apenas existe um hotel na aplicação em cada momento. Quando se abandona um hotel com modificações não guardadas (porque se cria ou abre outro), deve perguntar-se se quer guardar a informação actual antes de o abandonar, através de `Prompt.saveBeforeExit()` (a resposta é obtida invocando o método `readBoolean()` ou `Form.confirm()` da framework de interacção com o utilizador).

Note-se que a opção Abrir não suporta a leitura de ficheiros de texto (estes apenas são utilizados na inicialização da aplicação). A opção Sair **nunca** guarda o estado da aplicação, mesmo que existam alterações.

4.1.2 Avaciar Estado do Ano

O comando deve apresentar na saída o código para a estação actual: 0 para Primavera, 1 para Verão, 2 para Outono e 3 para Inverno

4.1.3 Ver Estado de Satisfação Global

O sistema apresenta, de imediato, o somatório do nível de satisfação de todas as entidades registadas no hotel: animais e funcionários. Note-se que o valor a apresentar é o que resulta do arredondamento ao inteiro mais próximo (via `Math.round`).

4.1.4 Gestão and consulta de dados da aplicação

Existem ainda no menu principal várias opções para abrir um segundo menu com as operações disponíveis sobre determinadas entidades do sistema (animais, funcionários, habitats and vacinas). Finais, e também disponibilizado uma opção para abrir o menu de consultas (pesquisas).

4.2 Animais Menu

Este menu permite visualizar alterar informação sobre animais. nas classes do package `hva.app.animal`: `DoShowAllAnimals`, `DoRegisterAnimal`,

`DoTransferToHabitat`, e `DoShowSatisfactionOfAnimal`.

4.2.1 Visualizar todos os animais

Esta opção permite visualizar todos os animais. O formato de apresentação de cada animal é o seguinte:

```
ANIMAL|idAnimal|nomeAnimal|idEspecie|historialDeSaude|idHabitat
```

O history of saúde and presentation of como descrito na secção 1.6.1, ie, uma sequência de eventos separados por vírgulas. Se o animal nunca foi vacinado (com ou sem sucesso), o historical saúde deve ser apresentado como `VOID`:

```
ANIMAL|idAnimal|nomeAnimal|idEspecie|VOID|idHabitat
```

4.2.2 Registrar um novo animal

Esta opção, ao registrar um novo animal. Se não existe, deve pedir-se o nome da espécie (`Prompt.speciesName()`) (cadeia de caracteres) e registra-la com o identificador indicado. , não se processando o registro.

4.2.3 Transferir um animal para um habitat

Esta opção, ao transfere um animal de um habitat para outro.

4.2.4 Calcular Satisfactory

Esta opção, ao apresentar a satisfação, ao de um animal .

4.3 Menu de Gestão de Funcionários

visualizar todos os funcionários; (ii) registrar um novo funcionário; (iii) atribuir uma nova responsabilidade a um funcionário; (iv) retirar uma responsabilidade a um funcionário;

Este menu permite visualizar, alterar informações sobre **funcionários** .

des This section: **Visualization todos**

Atribuir uma nova responsabilidade a um funcionário , **Retirar uma res-**

The function of the program is as follows : `DoShowAllEmployees`, `DoRegisterEmployee`, `DoAddResponsibility`,

`DoRemoveResponsibility` e `DoShowSatisfactionOfEmployee` .

4.3.1 Visualizar todos os funcionários

Esta opção, ao permitir visualizar todos os funcionários.

`tipo|id|nome|idResponsabilidades`

Os valores para o campo tipo são VET ou TRT. Se não tiver responsabilidades atribuídas, o formato de apresentação é o seguinte:

`tipo|id|nome`

4.3.2 Registrar um novo funcionário

Esta opção, ao permite registrar um novo funcionário.

(cadeia de caracteres) funcionário (`Prompt.employeeType()`). Valores, a pergunta referente ao tipo é repetida até se obter uma resposta válida. Quando um funcionário é registrado fica sem qualquer responsabilidade.

Se já existir um funcionário com o mesmo identificador, deve ser lançada a exceção `DuplicateEmployeeKeyException`, não se realizando qualquer ação.

4.3.3 Atribuir uma nova responsabilidade a um funcionário

Esta opção, ao atribui uma nova responsabilidade a um funcionário . ao identificador de um habitat, se for um tratador. Se o funcionário já tinha essa responsabilidade, não é executada nenhuma ação .

4.3.4 Retirar uma response a um funcionario

Esta opção retira uma responsabilidade a um funcionario . ou ao identificador de um habitat, se for um tratador .

4.3.5 Calcular satisfac,ao de um funcion ario

Esta opção apresenta a satisfac,ao de um funcion ario .

4.4 Menu of Habitats

Este menu permit visualizar alter informac,ao sobre funcion arios .

Alterar influência de um habitat sobre uma espécie ao

Visualizar

Plantar

classes do package hva.app.habitat: DoShowAllHabitats, DoRegisterHabitat, DoChangeHabitatArea, DoChangeHabitatInfluence, DoAddTreeToHabitat e DoShowAllTreesInHabitat.

4.4.1 Visualize todos os habitats

Esta opção permite visualizar todos os habitats. O formato de apresentac,ao para cada habitat éo seguinte:

HABITAT|idHabitat|nome|area|número Árvores

Esta linha inicial pode ser seguida de uma ou mais linhas, cada uma com a descriç,ao de cada árvore pertencente ao habitat usando o seguinte formato:

ÁRVORE|idÁrvore|nomeÁrvore|idadeÁrvore|dificuldadeBaseLimpeza|tipoÁrvore|cicloBiológico

O valor relativo ao campo dificuldade BaseLimpeza corresponde ao valor da dificuldade base de limpeza da árvore. Os valores possíveis para o campo tipo Árvore são PERENE e CADUCA.

Estac,ao	Inverno	Primavera	Verão	Outono
Tipo de Árvore				
Folha caduca	SEMFOLHAS	GERARFOLHAS	COMFOLHAS	LARGARFOLHAS
Folha perene	LARGARFOLHAS	GERARFOLHAS	COMFOLHAS	COMFOLHAS

4.4.2 Registrar um novo habitat

Esta opção regista um novo habitat. O sistema pede o identifiçador, o nome (Prompt.habitatName()) (cadeia de caracteres) e a área do habitat (Prompt.habitatArea ()) (numero inteiro) , sendo registado o novo habitat. acc,ao.

4.4.3 Alterar an area de um habitat

Esta opção altera an área de um habitat. O sistema pede o identificador do habitat assim como a nova área desse habitat (Prompt.habitatArea()) (numero inteiro).

4.4.4 Alterar influencia de um habitat sobre uma espécie

Esta opção altera influ ência de um habitat sobre uma espécie . resposta: POS), negativa (resposta: NEG), ou neutra (resposta: NEU).

This extreme pergunta é repetida at e se obter uma resposta válida.

4.4.5 Plantar uma nova arvore num habitat

Esta opção introduz uma nova arvore num habitat.), a dificuldade base de arvore a limpeza associada a arvore (Prompt.treeDifficulty()) (numero inteiro) eo tipo de arvore (Prompt.treeType) para arvore de folha perene; CAD para arvores de folha caduca), registado-se a nova arvore. Se a resposta relativa ao tipo n ao corresponder a nenhum dos dois valores, esta ultima pergunta e repetida at e se obter uma resposta valida. Se ja existir uma arvore com o mesmo identificador, lancada a excepção DuplicateTreeKeyException, nao se realizando qualquer accção. Caso seja criada uma arvore, ent ao deve apresent a-la utilizando o formato descrito em 4.4.1. ao esta opção,

4.4.6 Visualization of habitats

Esta opção permit visualizar todas as arvores um determinado habitat.

Esta opção apresenta todas as arvores de todos os habitats. O formato de apresentacção de cada e como descrito em 4.4.1. arvore

4.5 Menu of Vacinas

Este menu permit visualizar alter informacção sobre vacinas e vaccinacções concretizados nas classes do package hva.app.vaccine: DoShowVaccine, DoRegisterVaccine,

DoVaccinateAnimal e DoShowVaccinations.

4.5.1 Visualizar todas as vacinas

Esta opção apresenta todas as vacinas. O formato de apresentacção de cada vacina, uma por linha, eo seguinte:

VACINA|idVacina|nomeVacina|numeroAplicacções|espécies

O campo especies contem os identificadores, separados por vírgulas, das especies de animais que podem receber a vacina.

4.5.2 Registrar uma nova vacina

Esta opção regista uma nova vacina . caracteres) da vacina e os identity das especies que podem receber esta vacina (Prompt.listOfSpeciesKeys()) (lista Unidentifiable separados por vírgulas. Os espaços brancos sao irrelevantes nesta resposta). De seguida regista a nova vacina. Deve ser lancada a excepção DuplicateVaccineKeyException, se existir uma vacina com o mesmo identificador, nao se realizando qualquer accção.

UnknownSpeciesKeyException, nao sendo registada a nova vacina.

4.5.3 Vacinar um animal

Esta opção regista a vaccinacção de um animal. O sistema pede o identificador da vacina, o identificador do veterinario (Prompt.eo identificador do animal a vacinar . Se o veterinario n ao tiver permissao para ministrar a vacina deve-ser lancada a excepção VeterinarianNotAuthorizedException eo comando nao tem qualquer efeito. Se a vacina n ao for adequada ao animal, deve-ser apresentada uma mensagem de aviso (Message.wrongVaccine()). Note-se que, neste caso, a vaccinacção aconteceu e os danos para o animal development ter sido registados.

4.5.4 Listar o historico de vaccinacções

Esta opção lista o historico de vaccinacções ordenado pela ordem em que as vacinas foram aplicadas. O sistema lista todas as vacinas aplicadas, usando o seguinte formato:

REGISTO-VACINA|idVacina|idVeterinário|idEspécie

4.6 Menu of Consultations

A list of options available to you: **rinario** and **consultant vacinas dadas com incuria**

Consultant habitat, Consultant vaccinas, Animal, Consultative actos medicos, veterinary medicine - ao

Estas opc,oes j a est

hva.app.search: DoShowAnimalsInHabitat, DoShowMedicalActsOnAnimal, DoShowMedicalActsByVeterinarian parcialmente concretizados nas classes do package

e DoShowWrongVaccinations.

4.6.1 Consulting animais de um habitat

Esta opc,ao apresenta informac, ao relativa aos animais de um habitat.

4.6.2 Consulting services for animals

Esta opc,ao apresenta a informac, ao relativa as vaccinac, oes de um animal, ao e como indicado em 4.5.4.

4.6.3 Consulting with veterinary medicine

Esta opc,ao apresenta informac, ao relativa a vaccinac, oes feitas por um veterin ario. ao UnknownVeterinarianKeyException .

Caso contrario, apresenta todas as vacinas por ele realizadas (por ordem de aplicac, ao). O formato de apresentac,ao e como indicado em 4.5.4.

4.6.4 Consultar vacinas dadas com incuria

Esta opc,ao apresenta informac, ao relativa a vaccinac, problemas qes err oneas.

de saude aos animais (por ordem de aplicac, ao). O formato de apresentac, ao e como indicado em 4.5.4.

5 Leitura de Dados a Partir de Ficheiros Textuais

Por omissao, quando aplicac, ao cont ao comec,a, n

em nenhuma informac, ao do lado da raiz.

além das opc,oes de manipulac,ao de ficheiros descritas no menu principal, yvel iniciar exteriormente aplicac,ao com um e poss ficheiro de texto especificado propriedade Java com o nome import .

The object corresponds to the object. The process doesnot work, the process does not work, it does not exist, it does not exist, it does not exist, it does not exist, it does not work.

Sugere-se a utilizac,ao do m etodoString.split() para dividir uma cadeia de caracteres camp emos. Cada entrada do ficheiro

The composting process of the campos separados peloton caracter '|'.

```
ESP´ECIE|id|nome
´ARVORE|id|nome|idade|dificuldade|tipo
HABITAT|id|nome|area|id´Arvore1,...,id´ArvoreN
ANIMAL|id|nome|idEsp´ecie|idHabitat
TRATADOR|id|nome|idHabitat1,...,idHabitatN
VETERIN´ARIO|id|nome|idEsp´ecie1,...,idEsp´ecieN
VACINA|id|nome|idEsp´ecie1,...,idEsp´ecieN
```

Todos os campos referentes a uma arvore, animal ou esp ecie s ao obrigat orios. ve- terinario (corresponding em ambos os casos a uma lista de esp ecies) e opcional eso e apresentado caso a lista indicada n ao seja

vazia. Se a lista for vazia, entao n ultimo campo, nem o separador. ao se apresenta o

Para os outros tipos, o ultimo campo e opcional: para um habitat, o ultimo campo correspond as` arvores implantadasao habitat, para a um tratador, o ultimo campo corresponde aos habitats so responsabilidade do tratador e para uma vacina ou veterin ario, o Ultimo campo corresponds to a list of esp ecies associada a vacina or ao veterinary.

```

ESP'ECIE|C10|ave
ESP'ECIE|C8|mam'ífero
ESP'ECIE|C1|peixe
'ARVORE|T1|Pinheiro 1|5|20|PERENE
'ARVORE|T4|Pinheiro 4|60|20|PERENE
'ARVORE|T2|Oliveira|1200|10|PERENE
'ARVORE|T6|Figueira|5|10|CADUCA
'ARVORE|T3|Píatano|300|20|CADUCA
'ARVORE|P1|Píatano|100|20|CADUCA
'ARVORE|F30|Figueira|50|20|CADUCA
HABITAT|AR1|Aldeia dos Macaços|20|T3,P1,F30,T2
HABITAT|AR2|Floresta tropical|30|T1,T4,T6
HABITAT|AR3|Deserto|3
ANIMAL|A1|Alex|C10|AR1
ANIMAL|A2|Nemo|C1|AR2
ANIMAL|AA|Bobi|C8|AR1
ANIMAL|MA|Chita|C8|AR2
TRATADOR|M2SO4|John Figueiredo|AR2,AR3
TRATADOR|M2O|Rohit Figueiredo|AR1
TRATADOR|M24|Rohit Figueiredo
VETERINÁRIO|V2|Jorge Figueiredo|C10,C8,C1
VETERINÁRIO|V4|Filomena Figueiredo
VETERINÁRIO|VR|Abdul Figueiredo|C8
VACINA|V3|Tíetano|C8
VACINA|V200|Parasitas intestinais|C10,C8,C1
VACINA|V42172|Tíetano|C10
VACINA|Vexperimental|Gripe A1|C8

```

Note-se que o programa **nunca** produz ficheiros com este formato, apenas lê ficheiros com este formato.

6 Execução dos Programas e Testes Automáticos

The results are correct and the programs are correct.

- um com extensão `.in` que representa o utilizador, ou seja, vai conter os dados de entrada a serem processados pelo programa;
- um com extensão `.out` e que representa o resultado esperado da execução do programa para o teste em causa;
- um com extensão `.import` e que representa o estado inicial do sistema. Este ficheiro é opcional e pode não fazer parte de um teste.

Dado um teste constituído pelos ficheiros `test.import`, `test.in` e `test.out`, é possível verificar automaticamente o resultado `correct` programa.

equivalent `-cp` do comando `java`), para localizar as classes do programa, incluindo a que contém o método correspondente ao ponto de entrada da aplicação (`hva.app.App.main`). As propriedades são tratadas automaticamente pelo código de apoio.

```
java -cp -Dimport=test.import -Din=test.in -Dout=test.outhyp hva.app.App
```

Import, import, executar, test and semelhante, anterior, specific and specific import. produz o ficheiro de saída `test.outhyp`.

Devem ser iguais. A comparação pode ser feita com o comando:

```
diff -b test.out test.outhyp
```

Este comando não deve produzir qualquer resultado quando os ficheiros são iguais.